

Java Backend Architecture

Interview Series

Chapter 7.3

Maven Plugins

50+ Interview Q&A

Code Examples

Architecture Diagrams

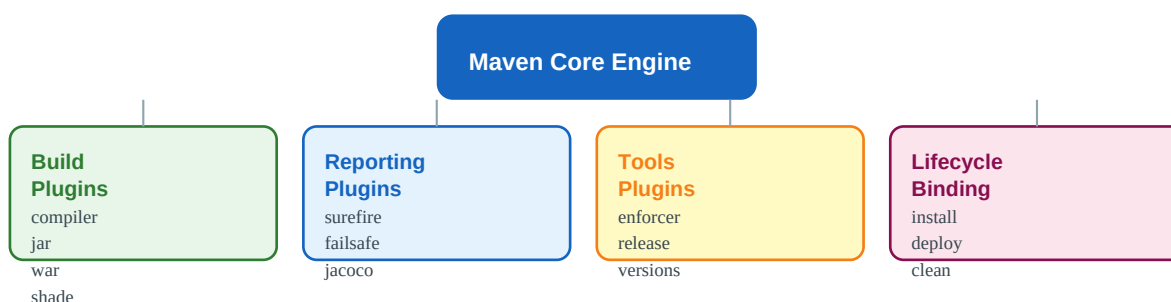
Advanced Topics

Chapter 7.3 — Maven Plugins

Introduction

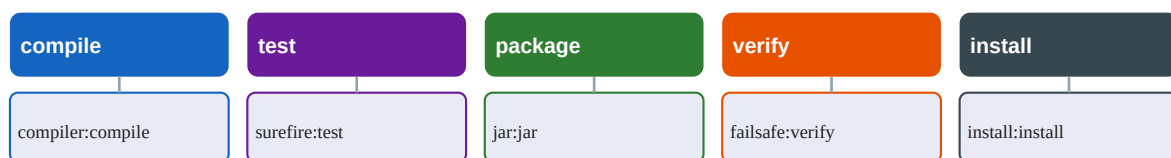
Maven plugins are the workhorses of the build system. Everything Maven does — compile, test, package, deploy — is implemented as a plugin. A plugin consists of one or more **Mojos** (Maven Old Java Objects), each implementing a single goal. Goals are bound to lifecycle phases (default bindings) or can be invoked directly. Understanding key plugins and how to configure them is essential for customising enterprise builds.

Plugin Architecture



Plugins bind Mojos (goals) to lifecycle phases. Each goal = one Java class implementing AbstractMojo.

Default Lifecycle Bindings



Default bindings for jar packaging — plugins automatically bound to lifecycle phases

Plugin Configuration

```
<build> <plugins> <plugin> <groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId> <version>3.12.1</version> <configuration>
<release>17</release> <compilerArgs> <arg>-parameters</arg> <!-- retain parameter names -->
</compilerArgs> <annotationProcessorPaths> <path> <groupId>org.mapstruct</groupId>
<artifactId>mapstruct-processor</artifactId> <version>1.5.5.Final</version> </path>
</annotationProcessorPaths> </configuration> </plugin> </plugins> </build>
```

maven-surefire-plugin (Unit Tests)

Runs unit tests during the **test** phase. Supports JUnit 4, JUnit 5 (Jupiter), and TestNG. For JUnit 5, you need the junit-platform-launcher on the classpath (or surefire 3.x auto-detects it).

```
<plugin> <groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId> <version>3.2.3</version> <configuration> <!--
```

```
Run tests in parallel --> <parallel>methods</parallel> <threadCount>4</threadCount> <!--
Include/Exclude patterns --> <includes> <include>**/*Test.java</include>
<include>**/*Tests.java</include> </includes> <excludes> <exclude>**/*IT.java</exclude>
</excludes> <!-- Pass system properties to tests --> <systemPropertyVariables>
<spring.profiles.active>test</spring.profiles.active> </systemPropertyVariables> <!-- Fail
build if no tests found --> <failIfNoTests>true</failIfNoTests> </configuration> </plugin>
```

maven-failsafe-plugin (Integration Tests)

```
<plugin> <groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-failsafe-plugin</artifactId> <version>3.2.3</version> <executions>
<execution> <goals> <goal>integration-test</goal> <!-- runs during integration-test phase -->
<goal>verify</goal> <!-- reports failures during verify phase --> </goals> </execution>
</executions> <configuration> <includes><include>**/*IT.java</include></includes>
</configuration> </plugin>
```

maven-shade-plugin (Uber-JAR)

```
<plugin> <groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-shade-plugin</artifactId> <version>3.5.1</version> <executions> <execution>
<phase>package</phase> <goals><goal>shade</goal></goals> <configuration>
<createDependencyReducedPom>true</createDependencyReducedPom> <!-- Relocate guava to avoid
classpath conflicts --> <relocations> <relocation> <pattern>com.google.common</pattern>
<shadedPattern>shaded.com.google.common</shadedPattern> </relocation> </relocations>
<transformers> <transformer
implementation="org.apache.maven.plugins.shade.resource.ManifestResourceTransformer">
<mainClass>com.example.Main</mainClass> </transformer> <!-- Merge META-INF/services for SPI
--> <transformer
implementation="org.apache.maven.plugins.shade.resource.ServicesResourceTransformer"/>
</transformers> </configuration> </execution> </executions> </plugin>
```

maven-enforcer-plugin

```
<plugin> <groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-enforcer-plugin</artifactId> <version>3.4.1</version> <executions>
<execution> <id>enforce</id> <goals><goal>enforce</goal></goals> <configuration> <rules>
<requireMavenVersion><version>[3.8,) </version></requireMavenVersion>
<requireJavaVersion><version>[17,) </version></requireJavaVersion> <dependencyConvergence/>
<requireReleaseDeps> <onlyWhenRelease>true</onlyWhenRelease> </requireReleaseDeps>
<bannedDependencies> <excludes> <exclude>commons-logging:commons-logging</exclude> </excludes>
</bannedDependencies> </rules> </configuration> </execution> </executions> </plugin>
```

maven-release-plugin

```
<!-- SCM configuration required --> <scm>
<connection>scm:git:https://github.com/example/my-app.git</connection>
<developerConnection>scm:git:https://github.com/example/my-app.git</developerConnection>
<url>https://github.com/example/my-app</url> </scm> <!-- Release steps: 1. mvn release:prepare
- Checks no uncommitted changes - Removes -SNAPSHOT from version - Tags in SCM - Bumps to next
development version 2. mvn release:perform - Checks out the tag - Builds and deploys the
release 3. mvn release:rollback (if something went wrong) --> # Dry run to verify what will
happen mvn release:prepare -DdryRun=true # Non-interactive release mvn release:prepare -B
-DreleaseVersion=2.0.0 -DdevelopmentVersion=2.1.0-SNAPSHOT
```

JaCoCo Plugin (Code Coverage)

```
<plugin> <groupId>org.jacoco</groupId> <artifactId>jacoco-maven-plugin</artifactId>
<version>0.8.11</version> <executions> <execution> <id>prepare-agent</id>
<goals><goal>prepare-agent</goal></goals> </execution> <execution> <id>report</id>
<phase>verify</phase> <goals><goal>report</goal></goals> </execution> <execution>
<id>check</id> <goals><goal>check</goal></goals> <configuration> <rules> <rule>
<element>BUNDLE</element> <limits> <limit> <counter>LINE</counter> <value>COVEREDRATIO</value>
<minimum>0.80</minimum> </limit> </limits> </rule> </rules> </configuration> </execution>
</executions> </plugin>
```

Key Plugins Reference Table

Plugin	Default Phase	Key Goal(s)	Purpose
maven-compiler-plugin	compile / test-compile	compile, testCompile	Compile Java sources
maven-surefire-plugin	test	test	Run unit tests
maven-failsafe-plugin	integration-test / verify	integration-test, verify	Run integration tests
maven-jar-plugin	package	jar	Create JAR artifact
maven-war-plugin	package	war	Create WAR artifact
maven-shade-plugin	package	shade	Create uber-JAR with dep relocation
maven-assembly-plugin	package	single	Create distribution archives
maven-install-plugin	install	install	Install artifact to local repo
maven-deploy-plugin	deploy	deploy	Upload artifact to remote repo
maven-clean-plugin	clean	clean	Delete build output directory
maven-enforcer-plugin	validate	enforce	Enforce project rules/constraints
maven-release-plugin	—	prepare, perform	Automate release cycle
jacoco-maven-plugin	verify	prepare-agent, report, check	Code coverage analysis
versions-maven-plugin	—	display-updates, set	Manage dependency/plugin versions
maven-source-plugin	verify	jar-no-fork	Package sources JAR
maven-javadoc-plugin	verify	jar	Generate Javadoc JAR
flatten-maven-plugin	process-resources	flatten	Resolve CI-friendly version placeholders

50+ Interview Questions & Answers

Q1. What is a Maven plugin?

A collection of Mojos (goals) that extend Maven's build capabilities. Everything Maven does is implemented as a plugin, including compiling, testing, packaging, and deploying.

Q2. What is a Mojo?

Maven Old Java Object — a Java class that implements `org.apache.maven.plugin.AbstractMojo` and represents a single goal. Each Mojo has a `@Mojo` annotation specifying its name and default lifecycle phase.

Q3. How do you invoke a plugin goal directly?

`mvn groupId:artifactId:version:goal` — e.g., `mvn org.apache.maven.plugins:maven-compiler-plugin:3.12.1:compile`. Or using the plugin prefix: `mvn`

compiler:compile.

Q4. What is a plugin prefix?

A shorthand alias for a plugin. maven-compiler-plugin → prefix 'compiler', maven-surefire-plugin → 'surefire'. Defined in plugin metadata and registered in Maven's plugin registry.

Q5. How are plugins different from dependencies?

Dependencies are artifacts used by your code at runtime/test. Plugins are tools that execute during the build process. Both are resolved from repositories but declared in different POM sections (vs).

Q6. What is the difference between and ?

activates the plugin for the current module. sets plugin version and configuration centrally (typically in parent POM) — child modules must still declare the plugin in to activate it.

Q7. What is the maven-compiler-plugin parameter?

Sets source version, target version, and bootstrap class path in one step for cross-compilation. Preferred over separate / since Java 9. Example: 17.

Q8. How do you add annotation processors in maven-compiler-plugin?

Use in the compiler plugin configuration. This is preferred over putting processors in as it keeps them off the compile classpath.

Q9. What does maven-surefire-plugin do?

Runs unit tests during the test phase. Supports JUnit 4, JUnit 5, and TestNG. Generates test reports in target/surefire-reports/.

Q10. How do you run a single test method with Surefire?

mvn test -Dtest=MyTest#myMethod. For multiple: -Dtest=MyTest#method1+method2. Supports wildcards: -Dtest=My*Test.

Q11. What is the difference between Surefire and Failsafe?

Surefire runs unit tests (test phase) and fails immediately on test failure. Failsafe runs integration tests (integration-test + verify phases) and is designed to continue even if tests fail, allowing post-integration-test teardown to run.

Q12. What does failsafe:integration-test do vs failsafe:verify?

integration-test runs the tests and records results. verify checks the recorded results and fails the build. This separation ensures teardown (post-integration-test) runs even when tests fail.

Q13. What naming convention does Failsafe expect?

By default: *IT.java, IT*.java, *ITCase.java. Surefire expects: *Test.java, Test*.java, *TestCase.java. Both are configurable via /.

Q14. What does maven-shade-plugin do?

Creates an uber-JAR by merging the project's compiled classes with all dependency JARs. Supports package relocation to avoid classpath conflicts and resource transformers for special merging.

Q15. What is package relocation in shade-plugin?

Renames package paths inside dependency classes: com.google.common → shaded.com.google.common. Prevents split-package issues when two libraries on the classpath use the same package names.

Q16. What is the ServicesResourceTransformer?

A shade plugin transformer that merges META-INF/services/* files from multiple JARs. Essential when dependencies use Java SPI and multiple JARs provide implementations for the same service interface.

Q17. What does the ManifestResourceTransformer do in shade-plugin?

Adds or modifies the MANIFEST.MF in the uber-JAR. Commonly used to set the Main-Class attribute for executable JARs.

Q18. What does createDependencyReducedPom do in shade-plugin?

Creates a dependency-reduced-pom.xml where shaded dependencies are removed, so consumers don't pull in already-bundled artifacts transitively.

Q19. What is maven-assembly-plugin?

Creates distribution packages (zip, tar.gz, tar.bz2) with custom content — project JAR, dependencies, scripts, config files. More flexible than shade but produces an archive, not an uber-JAR.

Q20. What is a predefined assembly descriptor?

Built-in descriptors like jar-with-dependencies, bin, src, project. The most common is jar-with-dependencies which creates a fat JAR similar to shade but without relocation support.

Q21. What does maven-enforcer-plugin do?

Enforces project constraints during the validate phase. Built-in rules cover Maven/JDK version requirements, dependency convergence, banned dependencies, required files, and more.

Q22. What is the DependencyConvergence rule?

Ensures all paths to the same artifact resolve to the same version — no version conflicts in the dependency tree. Fails the build if any artifact has multiple versions.

Q23. What is the requireReleaseDeps rule?

Fails the build if any dependency is a SNAPSHOT version. Usually activated with true so it only applies when the project version is a release (non-SNAPSHOT).

Q24. What does maven-release-plugin:prepare do?

Checks for uncommitted changes, removes -SNAPSHOT from version, runs the build to verify, creates a SCM tag, bumps to next development -SNAPSHOT version, and commits these changes.

Q25. What does maven-release-plugin:perform do?

Checks out the release tag from SCM, runs the build with the release version, and deploys the artifact to the remote repository.

Q26. How do you roll back a failed release?

mvn release:rollback — reverts pom.xml version changes and deletes the SCM tag if it was created. Use when release:prepare fails partway through.

Q27. What is JaCoCo and how is it integrated with Maven?

JaCoCo is a Java code coverage library. The jacoco-maven-plugin integrates it: prepare-agent sets the JVM agent for test execution, report generates HTML/XML coverage reports, check enforces minimum coverage thresholds.

Q28. How do you enforce minimum code coverage with JaCoCo?

Use the jacoco:check goal with defining (LINE, BRANCH, METHOD, etc.) and ratio. Bind it to the verify phase to fail builds that don't meet thresholds.

Q29. What does jacoco:prepare-agent do?

Sets the `argLine` JVM argument to attach the JaCoCo agent to the test JVM. The agent instruments bytecode and records which lines/branches are executed during tests.

Q30. What is the maven-source-plugin?

Packages project source files into a `-sources.jar`. Typically bound to the `verify` or `package` phase using the `jar-no-fork` goal. Published alongside the main JAR for IDE source attachment.

Q31. What is the maven-javadoc-plugin?

Generates Javadoc and packages it into a `-javadoc.jar`. The `jar` goal is typically bound to `verify` or `package`. Both sources and javadoc JARs are required for publishing to Maven Central.

Q32. What is the flatten-maven-plugin?

Resolves CI-friendly version placeholders (`${revision}`, `${sha1}`, `${changelist}`) in POM files before installing/deploying. Produces a resolved POM that consumers can parse without property context.

Q33. How do you configure plugin execution for a specific profile?

Define the configuration inside a section. The plugin is only active when the profile is activated: `mvn clean install -Pprod`.

Q34. What is the exec-maven-plugin?

Allows running Java programs or shell commands from Maven builds. `exec:java` runs a Java class in the same JVM. `exec:exec` spawns a new process. Useful for integration test setup or code generation.

Q35. How do you run a Java class with exec-maven-plugin?

`mvn exec:java -Dexec.mainClass=com.example.Main -Dexec.args='arg1 arg2'`. Or configure in `pom.xml` with and inside the plugin configuration.

Q36. What is the spring-boot-maven-plugin?

Spring Boot's build plugin. `bootJar`/`bootWar` creates an executable uber-JAR/WAR with an embedded container. `bootRun` starts the application. `build-image` creates an OCI image using Buildpacks.

Q37. What is the difference between maven-jar-plugin:jar and spring-boot-maven-plugin:repackage?

`jar:jar` creates a standard thin JAR. Spring Boot's `repackage` takes the thin JAR and repackages it into an executable fat JAR with a special `BOOT-INF` layout, embedding all dependencies.

Q38. What does the versions-maven-plugin:set goal do?

Updates the project version in `pom.xml` and all submodule POMs: `mvn versions:set -DnewVersion=2.0.0`. Creates `.versionsBackup` files. Run `versions:commit` to clean up backup files after review.

Q39. What is the checkstyle-maven-plugin?

Runs Checkstyle static analysis against Java source code to enforce coding style rules. Fails the build (when configured) if code violates the defined rule set (Google style, Sun checks, custom XML).

Q40. What is the PMD plugin?

Runs PMD and CPD (copy-paste detector) analysis. PMD detects code quality issues (unused variables, empty catch blocks, etc.). CPD detects duplicate code. Both can be configured to fail the build.

Q41. What is the SpotBugs plugin?

Successor to FindBugs — performs bytecode analysis to detect potential bugs (null dereference, resource leaks, bad practices). Run with `mvn spotbugs:check`. Integrates with OWASP for security rules.

Q42. What is the OWASP dependency-check plugin?

Scans project dependencies against the NVD (National Vulnerability Database) CVE list to identify known vulnerabilities: `mvn dependency-check:check`. Configurable CVSS score threshold for build failure.

Q43. How do you bind a plugin goal to a lifecycle phase?

In the plugin's element: `packageshade`. The goal runs when Maven processes the specified phase.

Q44. Can a plugin have multiple executions?

Yes. Define multiple elements each with a unique `id`, different `goals`, and/or different `phases`. Example: `generate-sources` and `generate-test-sources` phases in `build-helper-plugin`.

Q45. What is the build-helper-maven-plugin?

Adds extra source directories, resources, and test sources to the build. Useful for generated code (from ANTLR, JAXB, etc.) that lives in non-standard directories.

Q46. What is the maven-resources-plugin?

Copies resources from `src/main/resources` and `src/test/resources` to the build output. Supports resource filtering — replacing `${property}` placeholders in resource files at build time.

Q47. What is resource filtering in Maven?

The `maven-resources-plugin` can replace `${property}` placeholders in resource files with Maven property values. Enable with `true`. Be careful with binary files — only filter specific file types.

Q48. How do you exclude a file type from resource filtering?

Use in resources plugin config, or declare separate entries — one filtered (text files), one non-filtered (binary files like images, fonts, keystores).

Q49. What is the maven-war-plugin?

Packages the project as a WAR for deployment to a servlet container. Handles `WEB-INF/lib` (dependencies), `WEB-INF/classes` (compiled code), `src/main/webapp` (web resources), and `web.xml`.

Q50. What is the maven-ear-plugin?

Creates an Enterprise Archive (EAR) packaging Java EE applications with multiple modules (WAR, JAR, EJB-JAR) and an `application.xml` descriptor for deployment to application servers.

Q51. How do you create a plugin?

Create a Maven project with `packaging=maven-plugin`, depend on `maven-plugin-api` and `maven-plugin-annotations`. Write a class extending `AbstractMojo` with `@Mojo` annotation. Implement `execute()`. Build and deploy like any artifact.

Q52. What is the @Parameter annotation in plugin development?

Injects Maven model values or `pom.xml` configuration into Mojo fields. Example: `@Parameter(defaultValue='${project.version}', readonly=true)` injects the project version.

Q53. What is @Component in plugin development?

Injects Maven components (e.g., `MavenProject`, `ArtifactResolver`) into Mojo fields using Plexus IoC. Example: `@Component MavenProject project;`